

Data Science

Shahid Beheshti University

Spring 2025

Regression Challenge Assignment #5

Mohammad Hossein Mirghaderi
99222105

1 Theoretical

Q1. Feature Engineering Effectiveness

Feature engineering is often called a creative process because it's more of an art than a strict science. There's no fixed set of rules that works for every dataset. It requires intuition, domain knowledge, and experimentation to figure out how to best represent the underlying data for a model. You're essentially trying to find signals in the noise by combining or transforming features in novel ways. A feature that seems unimportant on its own might become incredibly predictive when combined with another.

For a team to tackle this effectively, collaboration is key. They should start with brainstorming sessions where people with different perspectives (e.g., data scientists, domain experts) can suggest ideas. It's crucial to document every new feature created—what it is, why it was made, and how it was calculated. Using version control for datasets and code also ensures that experiments are reproducible and that the team can easily track which changes led to improvements.

Q2. Categorical Encoding methods

(a) Comparison of Encoding Methods

One-Hot Encoding: This method creates a new binary (0 or 1) column for each unique category in a feature. It's straightforward but can lead to a huge number of columns if the feature has many categories.

Label Encoding: This simply converts each category into a unique integer (e.g., 'red' -> 0, 'green' -> 1, 'blue' -> 2). This is simple but can trick a model into thinking there's an ordinal relationship when there isn't one.

Target Encoding: This replaces each category with the average value of the target variable for that category. It's a powerful way to capture information about the target, but it's also risky because it can easily lead to overfitting if not handled carefully.

Hash Encoding: This uses a hash function to convert categories into a fixed number of numerical features. It's a good compromise for features with very high cardinality, as it controls the number of output dimensions, but it can lead to "collisions" where different categories are mapped to the same hash value.

(b) Limitations of One-Hot Encoding

The main drawback of one-hot encoding shows up when you have

high-cardinality features—categorical variables with many unique values (like zip codes or user IDs). This method creates a new column for every single category. If you have thousands of categories, you get thousands of new columns. This makes your dataset massive and sparse (mostly filled with zeros), which can make model training very slow and memory-intensive. This issue is often called the "curse of dimensionality."

(c) When to Use Label Encoding

Label encoding is most appropriate when the categorical feature has a clear

ordinal relationship. For example, if you have categories like 'low', 'medium', and 'high', it makes sense to encode them as 0, 1, and 2. This preserves the inherent order of the categories, which can be a useful signal for the model. Using one-hot encoding in this scenario would treat each category as independent and lose that valuable ordering information.

(d) Avoiding Overfitting with Target Encoding

To prevent overfitting (or data leakage) with target encoding, you must use a robust

cross-validation strategy. The key is to calculate the target means on your training data

only and then apply those means to your validation/test set. You should never use the target information from the data you're trying to predict. Another technique is to add smoothing, where the category's mean is blended with the overall mean of the target. This is especially helpful for rare categories that have very few data points, as their calculated means can be unreliable.

(e) Hash Encoding and Sparsity

Hash encoding addresses the sparsity problem by mapping a potentially large number of categories into a pre-defined, fixed number of columns. Instead of

creating one column per category like one-hot encoding, it uses a hash function to decide which of the new columns the category belongs to. The main potential issue it introduces is

hash collisions. This happens when two or more different original categories get mapped to the same hash value in the same column. This can confuse the model, as it might start treating distinct categories as if they were the same.

Q3. Category Embeddings

Category embeddings work by mapping each category in a feature to a dense, low-dimensional vector of continuous values. Think of it as giving each category a set of coordinates in a multi-dimensional space. These vectors (or "embeddings") aren't pre-defined; they are learned by a neural network during the training process.

They often outperform traditional methods for high-cardinality features because these learned vectors can capture complex relationships and similarities between categories. This is a much richer and more efficient representation than the sparse, high-dimensional output of one-hot encoding, which treats all categories as completely independent.

Q4. Rounding Numerical Features

The main benefit of rounding numerical features is that it can

reduce noise and act as a form of regularization. By grouping close-together values, you might prevent the model from overfitting to small, insignificant fluctuations in the data that don't generalize well.

The risk, however, is that you might

lose valuable information. If the rounding is too aggressive, you could smooth over important granular patterns in the data, potentially weakening your model's predictive power.

Q5. Non-linearities and Cyclical Features

(a) Solving Non-Linear Problems with Linear Models

Linear models can only capture straight-line relationships. However, you can help them model non-linear patterns by creating new features that are non-linear transformations of the original ones. For example, by creating

interaction features (e.g., $\text{feature_A} \times \text{feature_B}$) or **polynomial features** (e.g., feature_A^2), you explicitly give the model non-linear relationships to work with. The model can then find a linear relationship with these new engineered features, which corresponds to a non-linear relationship with the original data.

(b) Handling Cyclical Features

When dealing with cyclical data like time of day or day of the week, a simple numerical encoding is misleading. To ensure models handle these correctly, you can transform them to represent their cyclical nature. A common technique is to use trigonometric functions, mapping the variable onto a circle with

sine and cosine transformations.

Q6. TF-IDF and Dimensionality Reduction

In Natural Language Processing (NLP),

TF-IDF (Term Frequency-Inverse Document Frequency) is a statistical measure used to evaluate how important a word is to a document within a collection of documents. It helps filter out common words and gives more weight to words that are frequent in a specific document but rare across all documents, which are often the most informative words.

After applying TF-IDF, you often end up with a very high-dimensional dataset, where each unique word becomes a feature.

Dimensionality reduction techniques can improve model performance by projecting this data into a much smaller number of dimensions. This helps to reduce noise, makes training faster, and can often lead to better generalization by forcing the model to learn from the most significant underlying patterns in the text data.

Q7. Neural Networks and Feature Engineering

Neural Networks, especially Deep Learning models, generally require less manual feature engineering because they can learn relevant features automatically. Their layered architecture allows them to perform what's called

representation learning. The first layer might learn simple features, the next layer might combine those to learn more complex features, and so on, until the final layers can recognize very abstract concepts. This ability to build a hierarchy of increasingly complex features from raw data means data scientists don't have to spend as much time manually creating them, which is a major advantage over traditional machine learning models